Operation Analytics and Investigating Metric Spike

Job Data Analysis

```
create database project_3;
use project_3;
```

A. Job Reviewed Over Time: Write an SQL query to calculate the number of jobs reviewed per hour for each day in November 2020.

```
SELECT
   ds AS dates,
   ROUND((COUNT(job_id) / SUM(time_spent)) * 3600) AS 'Jobs_receives per hour'
FROM
   job_data
WHERE
   ds BETWEEN '11/1/2020' AND '11/30/2020'
GROUP BY ds;
```

B. Throughput Analysis: Write an SQL query to calculate the 7-day rolling average of throughput. Additionally, explain whether you prefer using the daily metric or the 7-day rolling average for throughput, and why.

```
SELECT
   ROUND(COUNT(event) / SUM(time_spent), 2) AS 'Weekly throughput'
FROM
   job_data;
```

```
SELECT
   ds AS dates,
   ROUND(COUNT(event) / SUM(time_spent), 2) AS 'daily throughput'
FROM
   job_data
GROUP BY ds
ORDER BY ds;
```

C. Language Share Analysis: Write an SQL query to calculate the percentage share of each language over the last 30 days.

```
SELECT
   language,
   COUNT(DISTINCT language),
   ROUND(100 * COUNT(*) / (SELECT
               COUNT(*)
            FROM
               job_data),
         2) AS percentage_share
FROM
   job_data
GROUP BY language
```

ORDER BY language DESC;

D. Duplicate Rows Detection: Write an SQL query to display duplicate rows from the job_data table.

```
SELECT

   actor_id, COUNT(*) AS duplicateuser_id

FROM

   job_data

GROUP BY actor_id

HAVING COUNT(*) > 1;
```

Investigating Metric Spike

```
use project_3;

CREATE TABLE events_1 (
   user_id INT8,
   occurred_at TEXT,
   event_type TEXT,
   event_name TEXT,
   location TEXT,
   device TEXT,
   user_type TEXT
);

LOAD DATA INFILE "events_1.csv" INTO TABLE events_1
FIELDS TERMINATED BY ','
IGNORE 1 LINES;
```

A. Weekly User Engagement: Write an SQL query to calculate the weekly user engagement.

```
SELECT
   EXTRACT(WEEK FROM occurred_at) AS week_num,
   COUNT(DISTINCT user_id) AS total
FROM
   events_1
GROUP BY week_num;
```

B. User Growth Analysis: Write an SQL query to calculate the user growth for the product.

```
select week_num, year_num, active_users,
sum(active_users) over (order by year_num, week_num rows between unbounded
preceding and current row) as user_growth
```

```
from (select extract(week from activated_at) as week_num,
extract(year from activated_at) as year_num,
count(distinct user_id) as active_users from users
where state = 'active'
group by year_num, week_num
order by year_num, week_num
) as ABC;
```

C. Weekly Retention Analysis: Write an SQL query to calculate the weekly retention of users based on their sign-up cohort.

```
SELECT
  EXTRACT(WEEK FROM occurred_at) AS week_num,
  EXTRACT(YEAR FROM occurred_at) AS year_num,
  device,
  COUNT(DISTINCT user_id) AS user_id
  FROM
  events_1
WHERE
  event_type = 'engagement'
GROUP BY 1 , 2 , 3
ORDER BY 1 , 2 , 3;
```

D. Weekly Engagement Per Device: Write an SQL query to calculate the weekly engagement per device.

```
SELECT
  device_name,
  AVG(num_users_using_device) AS avg_weekly_users,
  AVG(times_device_use_current_week) AS avg_times_used_weekly
FROM
  (SELECT
    WEEK(occurred_at) AS week,
      device AS device_name,
      COUNT(DISTINCT user_id) AS num_users_using_device,
      COUNT(device) AS times_device_use_current_week
  FROM
    events_1
  WHERE
    event_name = 'login'
  GROUP BY 1 , 2
  ORDER BY 1) a
GROUP BY 1
ORDER BY 2 DESC , 3 DESC;
```

E. Email Engagement Analysis: Write an SQL query to calculate the email engagement metrics.

```
select
week,
num_users,
```

```
time_weekly_digest_sent,
time_weekly_digest_sent-lag(time_weekly_digest_sent) over(order by week) as
time_weekly_digest_sent_growth,
time_email_open,
time_email_open-lag(time_email_open) over(order by week) as time_email_open_growth,
time_email_clickthrough,
time_email_clickthrough-lag(time_email_clickthrough) over(order by week) as
time_email_clickthrough_growth
from
(select week(occurred_at)as week,
count(distinct user_id) as num_users,
sum(if(action='sent_weekly_digest',1,0)) as time_weekly_digest_sent,
sum(if(action='email_open',1,0)) as time_email_open,
sum(if(action='email_clickthrough',1,0)) as time_email_clickthrough
 from email_events
 group by 1
 order by 1) as ABC;
```